Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «*Сервис-ориентированная архитектура*»

Отчёт
По лабораторной работе №2

Вариант: 21

Сдающие:
Петров Вячеслав Маркович
Елисеев Константин Иванович

группа P3408

Преподаватель:
Кривоносов Егор Дмитриевич

г. Санкт-Петербург, 2026 г.

Задание

На основе разработанной в рамках лабораторной работы #1 спецификации реализовать два веб-сервиса и использующее их API клиентское приложение.

Требования к реализации и развёртыванию сервисов:

- Первый ("вызываемый") веб-сервис должен быть реализован на фреймворке Spring MVC REST и развёрнут в окружении под управлением сервера приложений Tomcat.
- Второй веб-сервис должен быть реализован на фреймворке Spring MVC REST, развёрнут в окружении под управлением ещё одного экземпляра сервера приложений Tomcat и вызывать REST API первого сервиса.
- **Для обоих сервисов необходимо реализовать все функции, задокументированные в API, в строгом соответствии со спецификацией!**
- Доступ к обоим сервисам должен быть реализован по протоколу https с самоподписанным сертификатом сервера. Доступ к сервисам посредством http без шифрования должен быть запрещён.

Требования к клиентскому приложению:

- Клиентское приложение может быть написано на любом веб-фреймворке, который можно запустить на сервере helios.
- Приложение должно обеспечить полный набор возможностей, предоставляемых API обоих сервисов - включая сортировку, фильтрацию и постраничный вывод элементов коллекции.
- Приложение должно преобразовывать передаваемые сервисами данные в человеко-читаемый вид - параграф текста, таблицу и т.д.
- Клиентское приложение должно информировать пользователя об ошибках, возникающих на стороне сервисов, в частности, о том, что сервису были отправлены невалидные данные.

Оба веб-сервиса и клиентское приложение должны быть развёрнуты на сервере helios.

Выполнение

Исходный код

https://github.com/virdxk/ticket-api-openapi

- https://github.com/virdxk/ticket-api-openapi/tree/main/ticket-service - Ticket Service
- https://github.com/virdxk/ticket-api-openapi/tree/main/person-service - Person Service
- https://github.com/virdxk/ticket-api-openapi/tree/main/booking-service - Booking Service
- https://github.com/virdxk/ticket-api-openapi/tree/main/common - общий модуль: формат ошибок, фильтрация, сортировка, пагинация
- https://github.com/virdxk/ticket-api-openapi/tree/main/client - клиентское приложение
- https://github.com/virdxk/ticket-api-openapi/tree/main/deploy - скрипты развёртывания на helios

Реализация сервисов

Все три сервиса из спецификации первой лабораторной работы (Ticket, Person и Booking) реализованы на Spring Boot 3.5 (Spring MVC REST) и собираются в WAR-архивы. Интерфейсы контроллеров и модели запросов и ответов сгенерированы из YAML-спецификаций плагином openapi-generator-maven-plugin (генератор spring, режим interfaceOnly), поэтому пути, параметры, типы и ограничения полей совпадают со спецификацией. Booking Service вызывает API Ticket Service и Person Service через клиентов, сгенерированных из тех же спецификаций (генератор java, библиотека restclient).

Каждый сервис разделён на три слоя: контроллер реализует сгенерированный интерфейс и формирует HTTP-ответ, сервис содержит бизнес-логику и транзакции, репозиторий (Spring Data JPA) работает с базой PostgreSQL. Общий для всех сервисов код вынесен в модуль common:

- единый формат ошибки {message, time, details} и сопоставление ошибок с кодами 400, 404, 409, 422, 500 и 503;
- разбор параметров filter и sort, проверка противоречивых условий, постраничная выборка;

- строгая обработка JSON: неизвестные поля, неверные типы и нарушения ограничений возвращаются одним ответом 422 со списком всех ошибок в details.

Развёртывание

Каждый сервис развёрнут в отдельном экземпляре сервера приложений Tomcat 10.1.60 на helios. Booking Service обращается к Ticket Service и Person Service по https.

Порты Tomcat на helios закрыты для внешних подключений, поэтому доступ снаружи организован через веб-сервер se.ifmo.ru, раздающий каталог public_html. Для каждого сервиса в public_html создан каталог, в котором правило .htaccess передаёт все запросы скрипту-посреднику. Скрипт без изменений пересылает метод, путь, параметры и тело запроса в Tomcat соответствующего сервиса по https и возвращает ответ. Клиентское приложение размещено в каталоге public_html/lab2.

Компонент	Порт Tomcat	Публичный адрес
Ticket Service	41390	https://se.ifmo.ru/~s412939/ticket-service/tickets
Person Service	41391	https://se.ifmo.ru/~s412939/person-service/persons
Booking Service	41392	https://se.ifmo.ru/~s412939/booking-service/booking
Клиент	-	https://se.ifmo.ru/~s412939/lab2/
Swagger UI	-	https://se.ifmo.ru/~s412939/

Доступ по https

При первом развёртывании утилитой keytool создаётся самоподписанный сертификат сервера. В конфигурации server.xml каждого экземпляра Tomcat объявлен единственный коннектор с SSLEnabled="true", использующий этот сертификат; http-коннектора нет, поэтому запросы без шифрования отклоняются.

```
$ curl http://localhost:41390/tickets
Bad Request
This combination of host and port requires TLS.



$ curl https://localhost:41390/tickets
curl: (60) SSL certificate problem: self-signed certificate


$ curl --cacert ~/soa/server.crt -o /dev/null -w "%{http_code}"
https://localhost:41390/tickets
200
```

```
$ openssl x509 -in ~/soa/server.crt -noout -subject -issuer -ext subjectAltName
subject=CN = helios.cs.ifmo.ru
issuer=CN = helios.cs.ifmo.ru
X509v3 Subject Alternative Name:
    DNS:localhost, DNS:helios.cs.ifmo.ru, IP Address:127.0.0.1
```

Клиентское приложение

https://se.ifmo.ru/~s412939/lab2/

Клиент написан на HTML, CSS и JavaScript без сторонних библиотек и содержит три вкладки: билеты, люди и бронирования. Для каждой коллекции доступны все операции API: просмотр страницы с фильтрацией по любому полю, многоуровневой сортировкой и постраничным выводом, получение, создание, частичное обновление и удаление элемента, а также специальные операции: удаление любого билета заданного типа, продажа VIP-копии билета и отмена всех бронирований человека. Данные выводятся таблицами и карточками, ошибки сервисов показываются с кодом ответа, сообщением и списком нарушений по полям; отправленный запрос и полученный ответ можно раскрыть под каждым результатом.

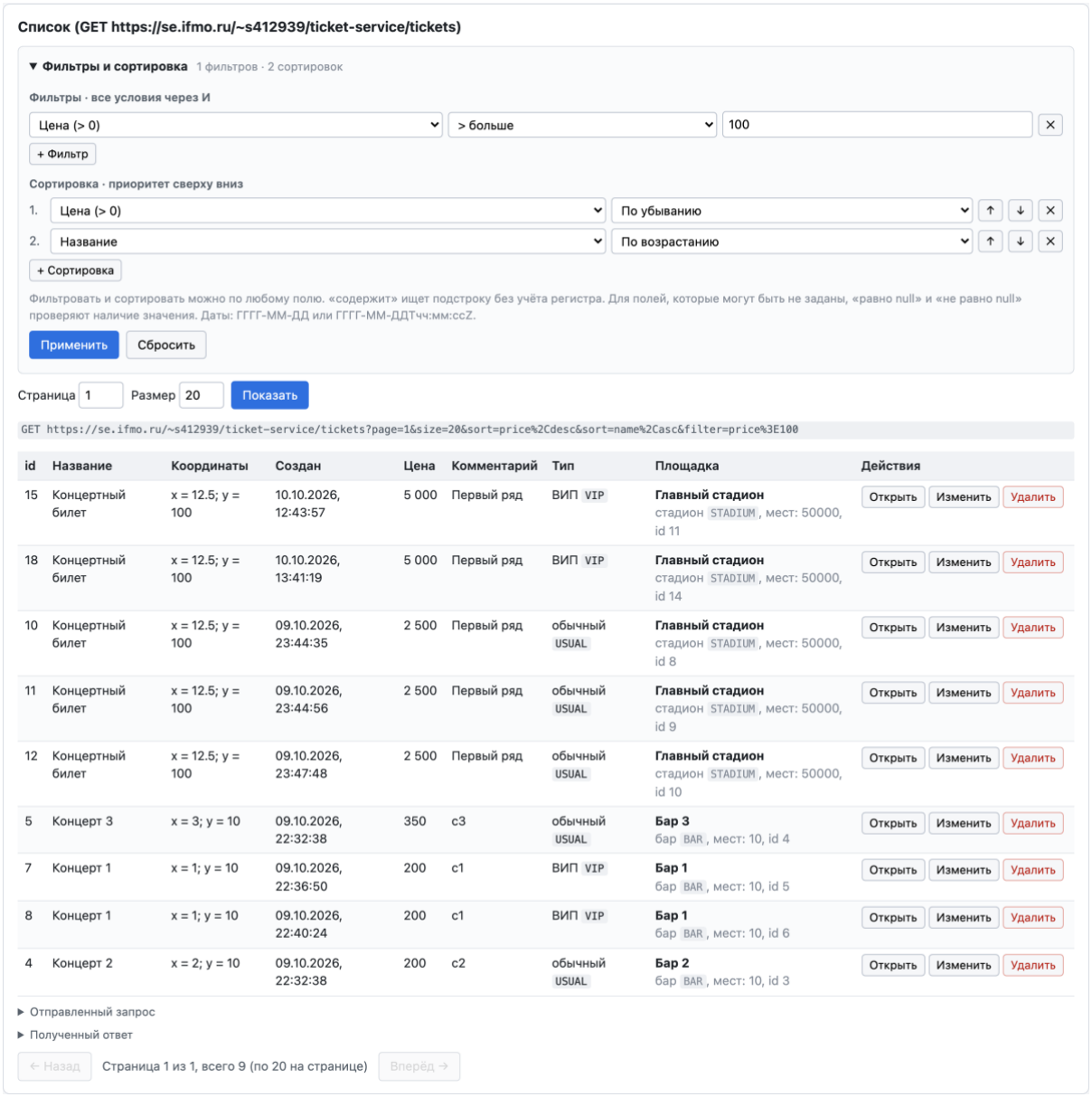


Рис 1. Список билетов с фильтрацией, сортировкой и постраничным выводом

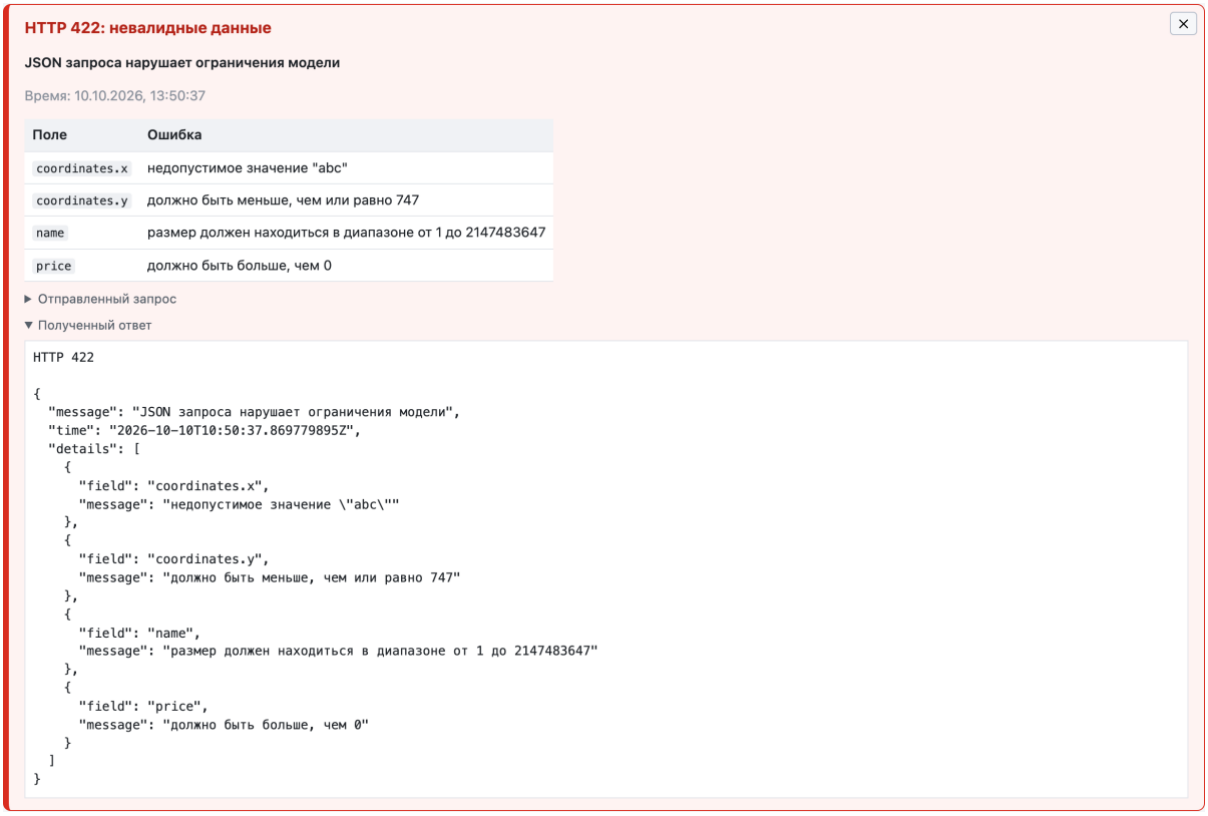


Рис 2. Ответ 422 на создание билета с невалидными данными



Рис 3. Продажа VIP-копии билета через Booking Service

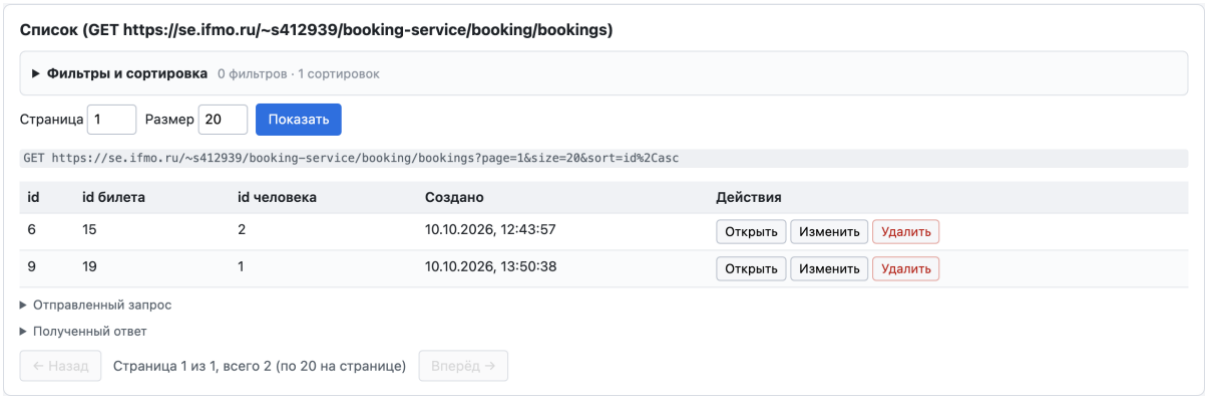


Рис 4. Список бронирований

Swagger UI

https://se.ifmo.ru/~s412939/

Интерактивная документация из первой лабораторной работы направлена на развёрнутые сервисы: в спецификациях указаны их публичные https-адреса, поэтому операции можно выполнять прямо из Swagger UI.

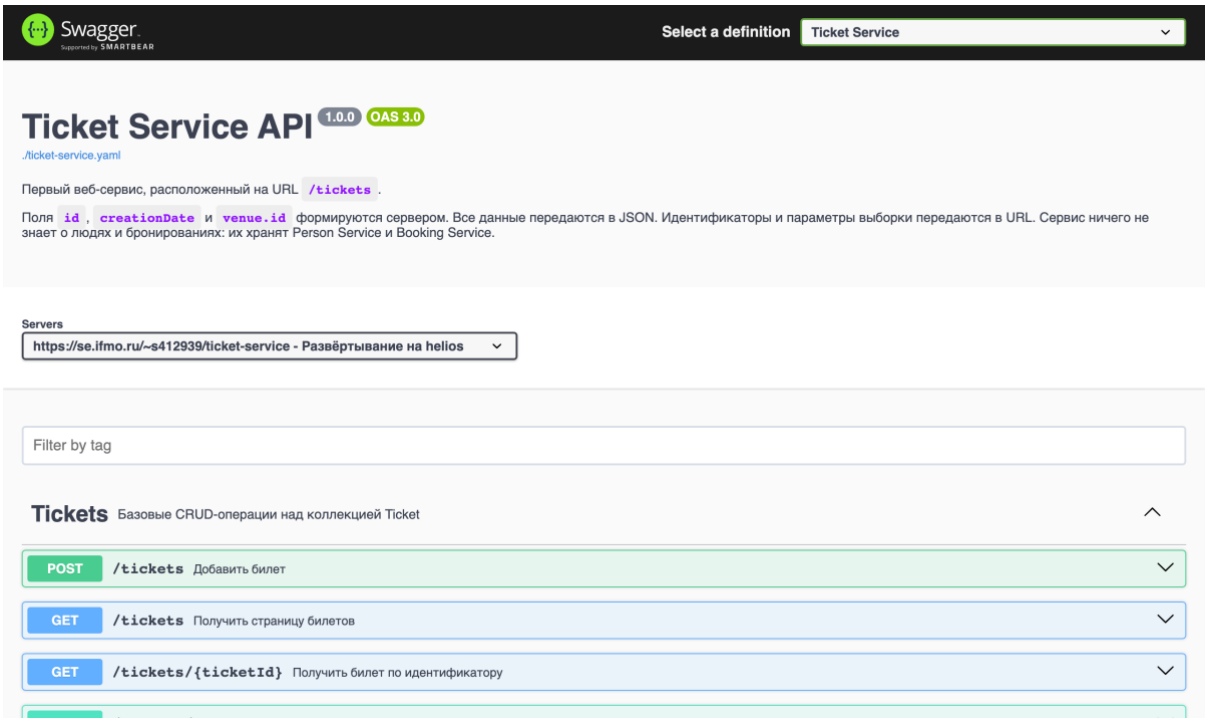


Рис 5. Swagger UI для Ticket Service

Выводы

Во время выполнения лабораторной работы по спецификации OpenAPI из первой лабораторной работы были реализованы три веб-сервиса на Spring MVC REST и клиентское приложение. Мы научились генерировать интерфейсы контроллеров, модели и REST-клиентов из спецификации с помощью openapi-generator, разделять сервис на контроллеры, сервисы и репозитории, реализовывать фильтрацию, сортировку и постраничную выборку, возвращать ошибки валидации в едином формате с кодами 400, 404, 409 и 422. Сервисы развёрнуты в отдельных экземплярах Tomcat на helios с доступом только по https и самоподписанным сертификатом, созданным утилитой keytool; для межсервисного взаимодействия настроено хранилище доверенных сертификатов.